NAME : P.HARSHITHA

BATCHNO: 05

ROLLNO: 2403A510C9

QUESTION1:

Implement a class for Course

- **Task 1: Let AI auto-complete constructor and attributes.**
- **Task 2: Use AI suggestions to add validation methods.**

PROMPT:

The user wants a Python class `Course`. I need to define its constructor and attributes, then add validation methods for these attributes.

TASK1:

For a `Course` class, typical attributes would include:

- `course_code`: A unique identifier for the course (e.g., "CS101", "MATH200").
- `title`: The full name of the course (e.g., "Introduction to Computer Science").
- `credits`: The number of academic credits the course is worth.
- `department`: The academic department offering the course.

TASK2:

CODE:

FINALTEST.py > ...

```python
import re

class Course:
    """
    Represents an academic course with attributes like code, title, credits, and department.
    Includes validation for each attribute.
    """

    def __init__(self, course_code: str, title: str, credits: float, department: str):
        """
        Initializes a new Course object.

        Args:
            course_code (str): The unique code for the course (e.g., "CS101").
            title (str): The full title of the course.
            credits (float): The number of academic credits for the course.
            department (str): The academic department offering the course.

        Raises:
            ValueError: If any of the provided attributes fail validation.
        """
        self._course_code = None  # Initialize to None to allow setter validation
        self._title = None
        self._credits = None
        self._department = None

        self.course_code = course_code
        self.title = title
        self.credits = credits
        self.department = department

    @property
    def course_code(self) -> str:
        """The unique code for the course."""
        return self._course_code

    @course_code.setter
    def course_code(self, value: str):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("Course code cannot be empty and must be a string.")
        # Optional: Add a regex pattern for course code format (e.g., "DEPT101")
        if not re.fullmatch(r'[A-Z]{2,4}\d{3,4}', value.strip().upper()):
            raise ValueError("Course code must follow a pattern like 'DEPT101' (e.g., 2-4 letters followed by 3-4 digits).")
        self._course_code = value.strip().upper()

    @property
    def title(self) -> str:
        """The full title of the course."""
        return self._title

    @title.setter
    def title(self, value: str):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("Course title cannot be empty and must be a string.")
        self._title = value.strip()
```

```python
    @property
    def credits(self) -> float:
        """The number of academic credits for the course."""
        return self._credits

    @credits.setter
    def credits(self, value: float):
        if not isinstance(value, (int, float)):
            raise ValueError("Credits must be a number.")
        if not 0.5 <= value <= 6.0: # Assuming credits are typically between 0.5 and 6.0
            raise ValueError("Credits must be between 0.5 and 6.0.")
        self._credits = float(value)

    @property
    def department(self) -> str:
        """The academic department offering the course."""
        return self._department

    @department.setter
    def department(self, value: str):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("Department cannot be empty and must be a string.")
        self._department = value.strip()

    def __str__(self):
        return (f"Course: {self.course_code} - {self.title} "
                f"({self.credits} credits, Dept: {self.department})")
```

```python
    def __repr__(self):
        return (f"Course(course_code='{self.course_code}', title='{self.title}', "
                f"credits={self.credits}, department='{self.department}')")

# Example Usage and Testing
if __name__ == "__main__":
    print("--- Testing Valid Course Creation ---")
    try:
        course1 = Course("CS101", "Introduction to Computer Science", 3.0, "Computer Science")
        print(course1)
        print(f"Course Code: {course1.course_code}")
        print(f"Title: {course1.title}")
        print(f"Credits: {course1.credits}")
        print(f"Department: {course1.department}")

        course2 = Course("MATH200", "Calculus I", 4, "Mathematics")
        print(course2)

        course3 = Course("ENG305", "Advanced Creative Writing", 3.5, "English")
        print(course3)

    except ValueError as e:
        print(f"Error creating valid course: {e}")

    print("\n--- Testing Invalid Course Creations ---")
```

```python
    try:
        Course("", "Invalid Course", 3.0, "Test Dept")
    except ValueError as e:
        print(f"Error: {e}")

    # Invalid course code (wrong format)
    try:
        Course("CS 101", "Invalid Course", 3.0, "Test Dept")
    except ValueError as e:
        print(f"Error: {e}")

    # Invalid title (empty)
    try:
        Course("PHY101", "", 3.0, "Physics")
    except ValueError as e:
        print(f"Error: {e}")

    # Invalid credits (negative)
    try:
        Course("BIO200", "Biology Basics", -1.0, "Biology")
    except ValueError as e:
        print(f"Error: {e}")

    # Invalid credits (too high)
    try:
        Course("ART400", "Art History", 7.0, "Art")
    except ValueError as e:
        print(f"Error: {e}")
```

```
    try:
        Course("CHEM100", "General Chemistry", "three", "Chemistry")
    except ValueError as e:
        print(f"Error: {e}")

    # Invalid department (empty)
    try:
        Course("HIST101", "World History", 3.0, "")
    except ValueError as e:
        print(f"Error: {e}")

    print("\n--- Testing Attribute Modification with Validation ---")
    try:
        course_to_modify = Course("PSY101", "Introduction to Psychology", 3.0, "Psychology")
        print(f"Original: {course_to_modify}")

        course_to_modify.credits = 4.0
        print(f"Modified credits: {course_to_modify}")

        course_to_modify.title = "Intro to Psych"
        print(f"Modified title: {course_to_modify}")

        # Attempt invalid modification
        course_to_modify.department = "  "
    except ValueError as e:
        print(f"Error during modification: {e}")

    try:
```

```
        course_to_modify.department = "  "
    except ValueError as e:
        print(f"Error during modification: {e}")

    try:
        course_to_modify.course_code = "invalid"
    except ValueError as e:
        print(f"Error during modification: {e}")
```

OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/FINALTEST.py
--- Testing Valid Course Creation ---
Course: CS101 - Introduction to Computer Science (3.0 credits, Dept: Computer Science)
Course Code: CS101
Title: Introduction to Computer Science
Credits: 3.0
Department: Computer Science
Course: MATH200 - Calculus I (4.0 credits, Dept: Mathematics)
Course: ENG305 - Advanced Creative Writing (3.5 credits, Dept: English)

--- Testing Invalid Course Creations ---
Error: Course code cannot be empty and must be a string.
Error: Course code must follow a pattern like 'DEPT101' (e.g., 2-4 letters followed by 3-4 digits).
Error: Course title cannot be empty and must be a string.
Error: Credits must be between 0.5 and 6.0.
Error: Credits must be between 0.5 and 6.0.
Error: Credits must be a number.
Error: Department cannot be empty and must be a string.

--- Testing Attribute Modification with Validation ---
Original: Course: PSY101 - Introduction to Psychology (3.0 credits, Dept: Psychology)
Modified credits: Course: PSY101 - Introduction to Psychology (4.0 credits, Dept: Psychology)
Modified title: Course: PSY101 - Intro to Psych (4.0 credits, Dept: Psychology)
Error during modification: Department cannot be empty and must be a string.
Error during modification: Course code must follow a pattern like 'DEPT101' (e.g., 2-4 letters followed by 3-4 digits).
PS C:\Users\pende\OneDrive\Desktop\wt2>
```

OBSERVATION:

The Python script defines a `Course` class that uses properties with setters to enforce robust validation for its attributes like `course_code`, `title`, `credits`, and `department`. This ensures data integrity by checking for correct types, formats, and ranges both when an object is created and when its attributes are modified later. The code is well-structured and includes a comprehensive set of tests to demonstrate its functionality and error handling.

QUESTION2:

Automated scheduling module

- **Task 1: Generate loop logic to detect timetable clashes.**
- **Task 2: Use AI completion to add exception handling**

PROMPT:

The user wants to build an automated scheduling module in Python.

TASK1:

- I need to augment the existing `Course` class to include scheduling information, specifically a `time_slot` (day, start time, end time).
- Then, I will implement a function that takes a list of these scheduled courses (a timetable).
- This function will use nested loops to compare every unique pair of courses in the list.
- The core logic inside the loops will check if two courses are on the same day and if their time intervals overlap. An overlap occurs if `start_time1 < end_time2` and `start_time2 < end_time1`.

CODE:

FINALTEST.py > ...

```python
import re
from typing import List, Tuple

# --- Task 2: Define a custom exception for handling clashes ---
class TimetableClashError(Exception):
    """Custom exception raised when a timetable clash is detected."""
    def __init__(self, course1, course2):
        message = (
            f"Timetable Clash Detected!\n"
            f"  - Course 1: {course1}\n"
            f"  - Course 2: {course2}"
        )
        super().__init__(message)

class Course:
    """
    Represents an academic course with validation and scheduling information.
    """
    VALID_DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]

    def __init__(self, course_code: str, title: str, credits: float, department: str, time_slot: Tuple[str, int, int]):
        """
        Initializes a new Course object.

        Args:
            course_code (str): The unique code for the course.
            title (str): The full title of the course.
            credits (float): The number of academic credits.
            department (str): The offering department.
            time_slot (Tuple[str, int, int]): A tuple of (Day, start_hour, end_hour).
                                              Hours are in 24-hour format (e.g., 9 for 9am, 14 for 2pm).
        """
```

```
        self._course_code = None
        self._title = None
        self._credits = None
        self._department = None
        self.course_code = course_code
        self.title = title
        self.credits = credits
        self.department = department

        # New attribute for scheduling
        self._time_slot = None
        self.time_slot = time_slot

    # --- Existing properties and setters (condensed for brevity) ---
    @property
    def course_code(self) -> str: return self._course_code
    @course_code.setter
    def course_code(self, value: str):
        if not isinstance(value, str) or not re.fullmatch(r'[A-Z]{2,4}\d{3,4}', value.strip().upper()):
            raise ValueError("Invalid course code format (e.g., 'CS101').")
        self._course_code = value.strip().upper()

    @property
    def title(self) -> str: return self._title
    @title.setter
    def title(self, value: str):
        if not isinstance(value, str) or not value.strip(): raise ValueError("Title cannot be empty.")
        self._title = value.strip()

    @property
```

```
    def credits(self) -> float: return self._credits
    @credits.setter
    def credits(self, value: float):
        if not isinstance(value, (int, float)) or not (0.5 <= value <= 6.0):
            raise ValueError("Credits must be a number between 0.5 and 6.0.")
        self._credits = float(value)

    @property
    def department(self) -> str: return self._department
    @department.setter
    def department(self, value: str):
        if not isinstance(value, str) or not value.strip(): raise ValueError("Department cannot be empty.")
        self._department = value.strip()

    # --- New property and setter for time_slot ---
    @property
    def time_slot(self) -> Tuple[str, int, int]:
        """The scheduled time slot for the course (Day, start_hour, end_hour)."""
        return self._time_slot

    @time_slot.setter
    def time_slot(self, value: Tuple[str, int, int]):
        if not isinstance(value, tuple) or len(value) != 3:
            raise ValueError("Time slot must be a tuple of (Day, start_hour, end_hour).")
        day, start_hour, end_hour = value
        if day not in self.VALID_DAYS:
            raise ValueError(f"Invalid day '{day}'. Must be one of {self.VALID_DAYS}.")
        if not (isinstance(start_hour, int) and isinstance(end_hour, int)):
            raise ValueError("Start and end times must be integers.")
        if not (0 <= start_hour < end_hour <= 23):
            raise ValueError("Invalid time range. Ensure 0 <= start_hour < end_hour <= 23.")
```

```python
        self._time_slot = value

    def __str__(self):
        day, start, end = self.time_slot
        return (f"{self.course_code}: {self.title} ({day} {start}:00-{end}:00)")

# --- Task 1: Generate loop logic to detect timetable clashes ---
def check_for_clashes(schedule: List[Course]):
    """
    Checks a list of courses for any scheduling clashes.

    Args:
        schedule (List[Course]): A list of Course objects representing the timetable.

    Raises:
        TimetableClashError: If a clash is detected between any two courses.
    """
    n = len(schedule)
    # Iterate through each unique pair of courses
    for i in range(n):
        for j in range(i + 1, n):
            course1 = schedule[i]
            course2 = schedule[j]

            day1, start1, end1 = course1.time_slot
            day2, start2, end2 = course2.time_slot

            # Check if courses are on the same day
            if day1 == day2:
```

```python
                if start1 < end2 and start2 < end1:
                    # --- Task 2: Raise the custom exception on clash ---
                    raise TimetableClashError(course1, course2)
    print("Success: No timetable clashes detected.")


# --- Example Usage and Testing ---
if __name__ == "__main__":
    print("--- Scenario 1: Valid Timetable (No Clashes) ---")
    try:
        schedule1 = [
            Course("CS101", "Intro to Python", 3.0, "CS", ("Monday", 9, 11)),
            Course("MATH200", "Calculus I", 4.0, "Math", ("Monday", 11, 13)),
            Course("ENG105", "Creative Writing", 3.0, "English", ("Tuesday", 10, 12))
        ]
        check_for_clashes(schedule1)
    except (ValueError, TimetableClashError) as e:
        print(f"Error: {e}")

    print("\n--- Scenario 2: Timetable with a Clear Clash ---")
    try:
        schedule2 = [
            Course("CS101", "Intro to Python", 3.0, "CS", ("Monday", 9, 11)),
            Course("PHY210", "Physics II", 4.0, "Physics", ("Wednesday", 14, 16)),
            Course("ART300", "Art History", 3.0, "Art", ("Monday", 10, 12)) # This clashes with CS101
        ]
        check_for_clashes(schedule2)
    # --- Task 2: Catch the specific TimetableClashError ---
    except TimetableClashError as e:
        print(f"Caught expected error:\n{e}")
    except ValueError as e:
```

```python
        print(f"Caught unexpected data error: {e}")


    print("\n--- Scenario 3: Timetable with Invalid Course Data ---")
    try:
        schedule3 = [
            Course("CS101", "Intro to Python", 3.0, "CS", ("Monday", 9, 11)),
            # Invalid day "Weekend"
            Course("BAD101", "Bad Data Course", 3.0, "Test", ("Weekend", 10, 12))
        ]
        check_for_clashes(schedule3)
    except ValueError as e:
        print(f"Caught expected data validation error: {e}")
    except TimetableClashError as e:
        print(f"Caught unexpected clash error: {e}")
```

OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/FINALTEST.py
--- Scenario 1: Valid Timetable (No Clashes) ---
Success: No timetable clashes detected.

--- Scenario 2: Timetable with a Clear Clash ---
Caught expected error:
Timetable Clash Detected!
  - Course 1: CS101: Intro to Python (Monday 9:00-11:00)
  - Course 2: ART300: Art History (Monday 10:00-12:00)

--- Scenario 3: Timetable with Invalid Course Data ---
Caught expected data validation error: Invalid day 'Weekend'. Must be one of ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'].
PS C:\Users\pende\OneDrive\Desktop\wt2>
```

OBSERVATION:

The `check_for_clashes` function efficiently detects timetable conflicts using nested loops and an accurate overlap condition. It leverages a custom `TimetableClashError` for specific, informative clash reporting, separating detection from error handling. The system is robust, handling both data validation (`ValueError`) and scheduling logic errors gracefully, with the `time_slot` attribute seamlessly integrated into the `Course` class.